飞腾嵌入式系统 CANopen 使用手册

（V1.0）

飞腾信息技术有限公司
www.phytium.com.cn
2025 年 08 月 18 日

版权所有 © 飞腾信息技术有限公司 2025。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区 https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台 https://service.phytium.com.cn/developer/

Phytium飞腾

目录

1 介绍 1
1.1 CANopenNode 和 CanFestival 对比 1
2 CANopenNode 移植和使用 1
2.1 CANopenNode 介绍 1
2.2 部署 CANopenLinux 2
2.3 CANopenNode 测试 2
2.3.1 CANopenDemo 下载和编译 3
2.3.2 运行 CANopenDemo 3
3 CanFestival 移植和使用 4
3.1 CanFestival 介绍 4
3.2 CanFestival 移植 5
3.3 CanFestival 测试 5
A 更新记录 7

Phytium飞腾

1 介绍

CANopen 协议由 CiA（CAN-in-Automation）组织维护，在 CAL(CAN Application Layer)的基础上发展而来。目前 CANopen 协议已经在运动控制、车辆工业、轨道交通、电机驱动、工程机械、船舶海运等行业得到广泛的应用。CANopen 协议详细内容参见：https://www.can-cia.org/can-knowledge/canopen。

在开源社区中，CANopenNode 和 CanFestival 是两个较为常用的 CANopen 协议栈。本文档主要介绍在飞腾嵌入式 Linux 系统下移植使用 CANopenNode 和 CanFestival。

1.1 CANopenNode 和 CanFestival 对比

CANopenNode 和 CanFestival 都能够用于嵌入式设备或 PC 上的 CANopen 应用开发。以下是两者的差异：

特性	CANopenNode	CanFestival
兼容性	支持 CiA 301、部分 CiA 302/305 扩展，主要实现从站功能	主要实现 CiA 301，实现了主/从站功能
项目活跃度	仍在持续维护和更新	基本无更新维护
配套开发工具	项目文档、工具丰富，便于快速搭建 CANopen 应用	文档、工具较少，且未更新

根据项目需求，若是开发 CANopen 从站应用，需要工业级长期支持，推荐 CANopenNode；若是用于 CANopen 主站开发，可以使用 CanFestival。

2 CANopenNode 移植和使用

2.1 CANopenNode 介绍

CANopenNode 是一个开源的 CANopen 协议实现，目标是为嵌入式设备和 PC 上的 CANopen 应用提供一个轻量、可移植、可扩展的解决方案。CANopenLinux 是在 Linux 系统上基于 CANopenNode 和 SocketCAN 的一个 CANopen 实现，它把 CANopen 协议封装成一个 Linux 用户态应用，方便在 PC 或嵌入式 Linux 环境下调试、部署 CANopen 设备功能。在 Linux 系统下主要通过 CANopenLinux 来实现基于 CANopenNode 的 CANopen 应用。

CANopenNode 项目地址：https://github.com/CANopenNode/CANopenNode

CANopenLinux 项目地址：https://github.com/CANopenNode/CANopenLinux

CANopenNode 官方文档地址：https://canopennode.github.io

2.2 部署 CANopenLinux

本文基于 E2000Q Demo 开发板移植和使用 CANopenNode 协议栈。

1) 在开发板上安装用户空间 CAN 工具集（can-utils），该工具集提供如 candump、cansend 等命令行工具，用于监控和调试 CAN 数据帧。

```
sudo apt-get install can-utils
```

2) 在开发板上克隆 CANopenLinux 项目代码，并更新子模块 CANopenNode 的代码。

```
git clone https://github.com/CANopenNode/CANopenLinux.git
cd CANopenLinux
git submodule init
git submodule update
```

3) 在开发板上编译 canopend，通过 canopend 可以将 Linux 主机变成 CANopen 节点设备。

```
cd CANopenLinux
make
sudo make install
```

4) 在开发板上编译 cocomm，cocomm 是配套 canopend 使用的命令行交互工具，主要用于在 Linux 下通过 SDO、NMT、LSS 来配置 CANopen 节点，方便调试和测试。可以理解为它是一个轻量级的 CANopen 控制台客户端。

```
cd CANopenLinux/cocomm
make
sudo make install
```

5) 在 Windows 主机中下载 CANopenEditor 应用程序（可选），CANopenEditor 是 CANopenNode 项目配套的一个图形化对象字典（Object Dictionary）编辑工具，主要用来方便地创建、查看和修改 CANopen 设备的对象字典。CANopenEditor 的下载地址为：https://github.com/CANopenNode/CANopenEditor/releases

2.3 CANopenNode 测试

CANopenDemo 是 CANopenNode 项目提供的一个示例应用程序，它的主要目的是让开发者快速理解 CANopenNode 协议栈在真实环境中的初始化、运行流程，以及如何和对象字典（OD）及应用层逻辑结合。本文使用 CANopenDemo 来测试 CANopenNode。

2.3.1 CANopenDemo 下载和编译

1) 克隆 CANopenDemo 的项目代码，并更新子模块的代码。

```
git clone https://github.com/CANopenNode/CANopenDemo.git
cd CANopenDemo
git submodule update --init –recursive
```

2) 编译 CANopen demoLinuxDevice，demoLinuxDevice 可以生成一个标准的 CANopen 节点，它包含对象字典，其中配置了常见的通信参数以及一些额外的制造商特定参数。可以使用 CANOpenEditor 编辑 demoLinuxDevice 的对象字典，默认的对象字典参见 demo/demoDevice.md。

```
cd CANopenDemo/demo
make
```

2.3.2 运行 CANopenDemo

1) 将 E2000Q Demo 开发板的 CAN0 和 CAN1 连接，将 CAN0、CAN1 连接在一个 CAN 网络上，如下图：

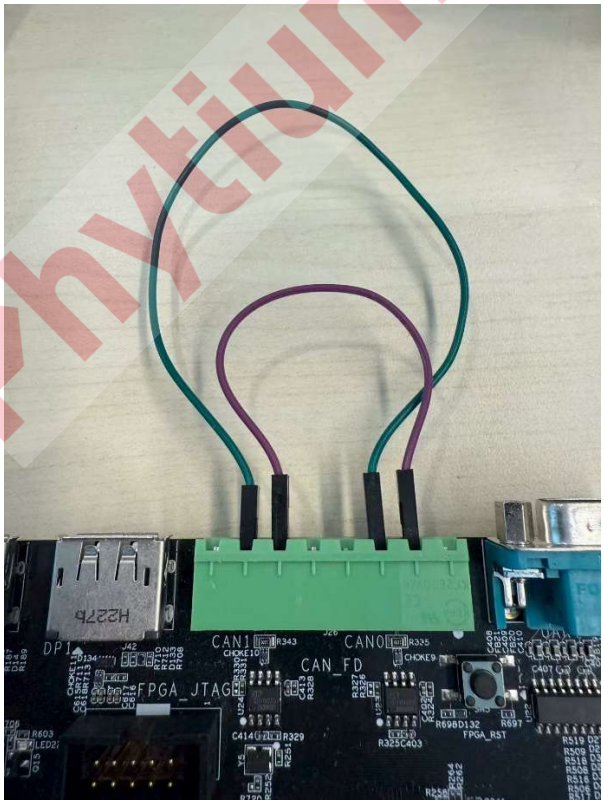


图 2.1 CAN0 和 CAN1 连接

2) 将开发板上电，设置 CAN0 和 CAN1 的波特率，并打开 CAN0、CAN1。

```
ip link set can0 type can bitrate 500000 fd off
ip link set can1 type can bitrate 500000 fd off
ip link set can0 up
ip link set can1 up
```

3) 单独打开一个终端通过 ssh 连接到开发板，运行 candump，方便监视 CANopen 协议数据。

```
candump -td can0
```

4) 运行测试程序：在 CAN0 上运行 canopend，生成一个 CANopen 节点，其 CANopen 节点 ID 为 1；在 CAN1 上运行 demoLinuxDevice，生成另一个 CANopen 节点，其 CANopen 节点 ID 为 4，CANopen 节点如图 2.2 所示：

```
canopend can0 -i 1 -c "/tmp/CO_command_socket"
demoLinuxDevice can1
```

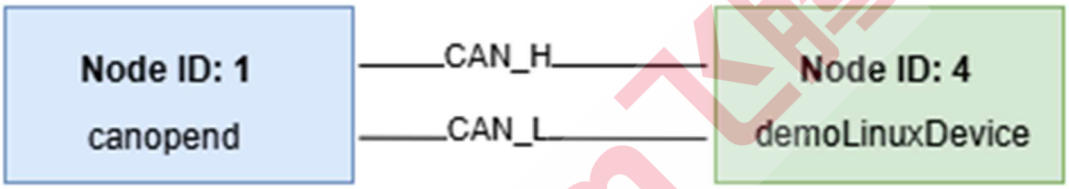


图 2.2 CANopen 节点示意图

5) 当 canopend 和 demoLinuxDevice 运行起来后，两个 CANopen 节点会发送上线报文，即 0x701 和 0x704 的 CAN ID 报文，在 candump 中能看到上线报文即表示环境搭建没问题，如图 2.3。具体的 CANopenNode 协议栈功能测试可以参考 CANopenDemo 项目中 tutorial 目录下的文档。

```
(408.957729)  can0  701   [1]  00
(000.049731)  can0  081   [8]  00 50 01 2F 14 00 00 00
(002.449016)  can0  704   [1]  00
(000.000241)  can0  084   [8]  00 50 01 2F 74 00 00 00
```

图 2.3 CANopen 节点上线报文

3 CanFestival 移植和使用

3.1 CanFestival 介绍

CanFestival 是一个开源的 CANopen 协议栈，用 C 语言实现，可为 PC、实时 IPC 和微控制器构建 CANopen 的主节点或从节点。

CanFestival 官网地址： https://canfestival.org

CanFestival 源码地址： https://hg.beremiz.org/canfestiva

3.2 CanFestival 移植

本文采用交叉编译的方式来移植 CanFestival，编译主机的环境为 X86 Ubuntu22.04。在 E2000Q Demo 开发板上对 CanFestival 进行功能测试。

1) 获取 CanFestival 源码，进行编译配置。

```
wget https://hg.beremiz.org/canfestival/archive/tip.tar.bz2
tar -xvf tip.tar.bz2
cd canfestival-de1fc3261f21/
./configure --cc=/opt/gcc-linaro-7.5.0-2019.12-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-gcc \
--arch=arm64 \
--os=unix \
--prefix=./install \
--target=unix \
--can=socket \
--timers=unix \
--debug=WAR,MSG
```

2) 编译并安装 CanFestival，将源码目录 install 文件夹下的所有内容拷贝到开发板文件系统上。

```
make
make install
```

3.3 CanFestival 测试

使用 CanFestival 项目 examples 中的 TestMasterSlave 例程在 E2000Q Demo 开发板上进行 CANopen 主从站通信测试，TestMasterSlave 这个例程主要展示了主从节点之间的相互通信，涉及 NMT、SDO、PDO、SYNC 等内容。设置开发板 CAN0 作为从站，从站 CANopen 节点 ID 为 2；CAN1 作为主站，主站 CANopen 节点 ID 为 1，通信波特率为 250K。具体步骤如下：

1) 编译 TestMasterSlave 测试例程，进入到源码目录 examples/TestMasterSlave 下，直接 make 编译，将编译出来的 TestMasterSlave 执行文件拷贝到开发板上。

2) 将 E2000Q Demo 开发板的 CAN0 和 CAN1 连接，将 CAN0、CAN1 连接在一个 CAN 网络上，参照 2.3.2。

3) 将开发板上电，设置 CAN0 和 CAN1 的波特率，并打开 CAN0、CAN1。

```
ip link set can0 type can bitrate 500000 fd off
ip link set can1 type can bitrate 500000 fd off
```

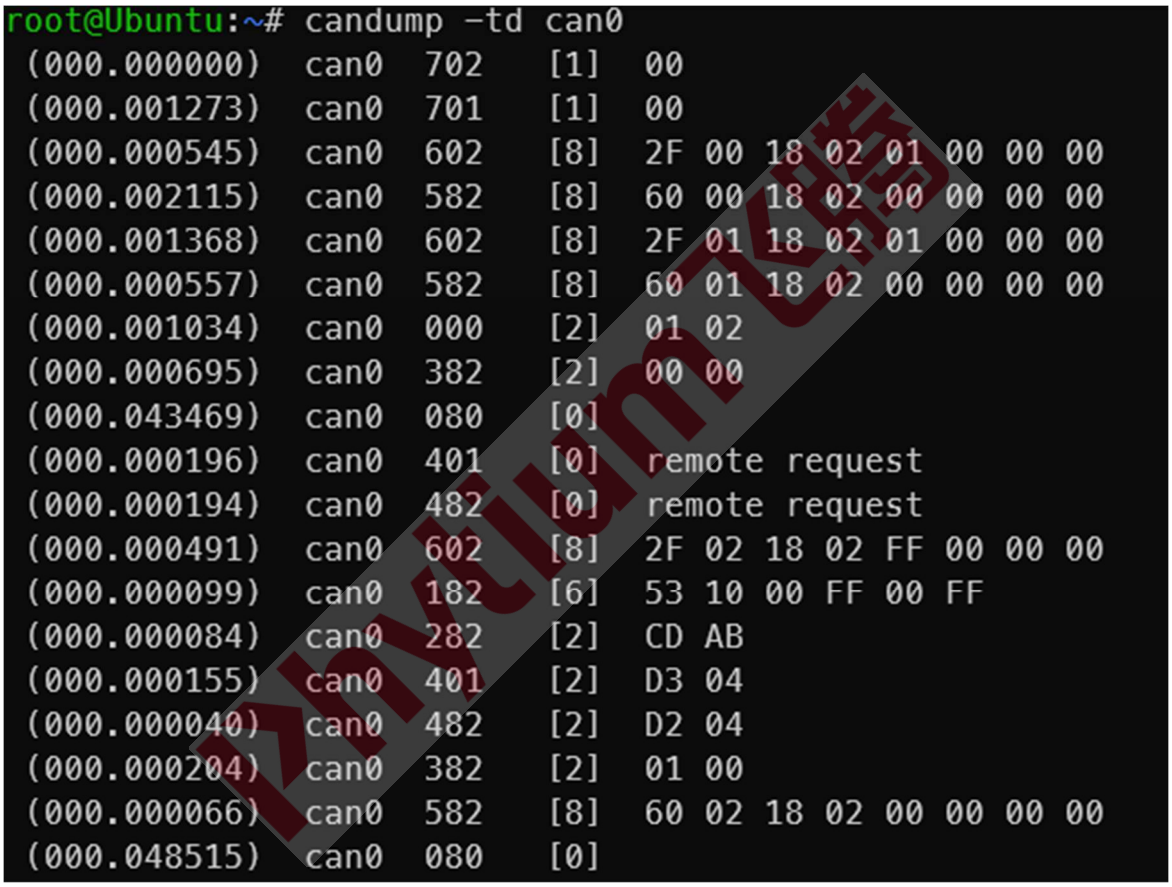
```
ip link set can0 up
ip link set can1 up
```

4) 单独打开一个终端通过 ssh 连接到开发板，运行 candump，方便监视 CANopen 协议数据。

```
candump -td can0
```

5) 在开发板上运行 TestMasterSlave 例程，-l 参数指定 libcanfestival_can_socket.so 库的位置，CAN0 作为从站，CAN1 作为主站。运行后在 candump 中能看到 CANopen 节点的交互报文数据。

```
./TestMasterSlave -l path/to/libcanfestival_can_socket.so -s 0 -S 250K -m 1 -M 250K
```



```
root@Ubuntu:~# candump -td can0
 (000.000000)  can0  702   [1]  00
 (000.001273)  can0  701   [1]  00
 (000.000545)  can0  602   [8]  2F 00 18 02 01 00 00 00
 (000.002115)  can0  582   [8]  60 00 18 02 00 00 00 00
 (000.001368)  can0  602   [8]  2F 01 18 02 01 00 00 00
 (000.000557)  can0  582   [8]  60 01 18 02 00 00 00 00
 (000.001034)  can0  000   [2]  01 02
 (000.000695)  can0  382   [2]  00 00
 (000.043469)  can0  080   [0]
 (000.000196)  can0  401   [0]  remote request
 (000.000194)  can0  482   [0]  remote request
 (000.000491)  can0  602   [8]  2F 02 18 02 FF 00 00 00
 (000.000099)  can0  182   [6]  53 10 00 FF 00 FF
 (000.000084)  can0  282   [2]  CD AB
 (000.000155)  can0  401   [2]  D3 04
 (000.000040)  can0  482   [2]  D2 04
 (000.000204)  can0  382   [2]  01 00
 (000.000066)  can0  582   [8]  60 02 18 02 00 00 00 00
 (000.048515)  can0  080   [0]
```

图 3.1 CANopen 节点交互报文数据

A 更新记录

发布日期	说明
2025-08-18	初次发布

Phytium飞腾